# 5.1 Bridges 아키텍처 및 구조 분석

## 개요

Polkadot SDK의 Bridges 컴포넌트는 서로 다른 블록체인 간의 신뢰할 수 있는 연결을 구축하기 위한 포괄적인 프레임워크입니다. 이 시스템은 Substrate 기반 체인, 파라체인, 그리고 이더리움과 같은 외부 블록체인 간의 크로스체인 통신을 가능하게 합니다.

Bridges는 온체인 컴포넌트(팔렛)와 오프체인 컴포넌트(릴레이어)로 구성되어 있으며, GRANDPA 합의 메커니즘을 기반으로 한 라이트 클라이언트 기술을 활용하여 신뢰성을 보장합니다.

## /bridges 디렉토리 구조 상세 분석

### 최상위 디렉토리 구조

```
bridges/
├── bin/ # 브릿지 노드 실행 파일
├── chains/ # 체인별 설정 및 프리미티브
├── docs/ # 브릿지 관련 문서
├── modules/ # 브릿지 팔렛 모음
├── primitives/ # 공통 프리미티브 및 타입 정의
├── relays/ # 릴레이어 구현체
├── snowbridge/ # 이더리움 브릿지 (Snowbridge)
├── testing/ # 테스트 유틸리티
├── README.md # 프로젝트 개요
```

```
├── CODE_OF_CONDUCT.md # 행동 강령
├── LICENSE # 라이선스
└── SECURITY.md # 보안 정책
```

## 1. modules/ - 브릿지 팔렛 시스템

브릿지의 핵심 온체인 컴포넌트들이 위치한 디렉토리입니다:

```
modules/
├── beefy/ # BEEFY 라이트 클라이언트 (개발 중)
├── grandpa/ # GRANDPA 라이트 클라이언트
├── messages/ # 크로스체인 메시지 전달
├── parachains/ # 파라체인 라이트 클라이언트
├── relayers/ # 릴레이어 보상 레지스트리
├── xcm-bridge-hub/ # 다중 동적 브릿지 지원
└── xcm-bridge-hub-router/ # XCM 브릿지 허브 라우터
```

## 2. relays/ - 릴레이어 시스템

오프체인 릴레이어 구현체들이 위치한 디렉토리입니다:

```
relays/
├── client-substrate/ # Substrate 클라이언트 인터페이스
├── equivocation/ # 이중 서명 감지 및 처리
├── finality/ # 최종성 릴레이어
```

```
├── lib-substrate-relay/ # Substrate 릴레이 라이브러리
├── messages/ # 메시지 릴레이어
├── parachains/ # 파라체인 릴레이어
└── utils/ # 공통 유틸리티
```

### 3. primitives/ - 공통 프리미티브

브릿지 시스템 전반에서 사용되는 공통 타입과 트레이트들이 정의된 디렉토리입니다.

### 4. snowbridge/ - 이더리움 브릿지

이더리움과 Polkadot 간의 신뢰할 수 있는 브릿지인 Snowbridge의 구현체가 위치한 디렉토리입니다.

## 브릿지 팔렛 상세 분석

### 5.1.1 GRANDPA 팔렛 (modules/grandpa/)

**목적**: 브릿지된 체인의 GRANDPA 최종성 가젯을 위한 라이트 클라이언트

**핵심 기능**:

- 브릿지된 체인의 헤더와 GRANDPA 최종성 증명(justification) 가져오기
- 검증자 세트 추적 및 업데이트
- 스토리지 증명 검증을 위한 최종화된 헤더 제공

**주요 특징**:

- **선택적 헤더 가져오기**: 모든 헤더를 가져올 필요 없이 필요한 헤더만 선택적으로 가져올 수 있음
- **필수 헤더 처리**: 검증자 세트가 변경되는 헤더는 필수적으로 가져와야 하며, 이러한 헤더 제출은 수수료가 면제됨

- **라이트 클라이언트 최적화**: 전체 체인 상태를 유지하지 않고도 최종성을 검증할 수 있음

**핵심 엔트리포인트**:

```
// 최종성 증명 제출
submit_finality_proof_ex(
finalized_header: Header,
justification: GrandpaJustification,
authority_set_id: SetId
)
```

### 5.1.2 Messages 팔렛 (modules/messages/)

**목적**: 소스 체인에서 타겟 체인으로 메시지를 전달하는 단방향 채널 제공

**핵심 개념**:

- **메시지 레인**: 4바이트 식별자로 구분되는 단방향 통신 채널
- **논스(Nonce)**: 각 레인 내에서 메시지에 할당되는 고유한 증가 정수
- **순서 보장**: 같은 레인을 통해 전송된 메시지는 전송 순서대로 전달됨

**메시지 생명주기**:

1. **Undelivered**: `send_message()` 호출로 메시지가 큐에 저장됨
2. **Delivered**: 릴레이어가 `receive_messages_proof()` 트랜잭션으로 메시지를 타겟 체인에 전달
3. **Confirmed**: 릴레이어가 `receive_messages_delivery_proof()` 트랜잭션으로 전달을 소스 체인에 확인

**주요 엔트리포인트**:

```
// 메시지 전송 (런타임 내부 호출)
send_message(lane_id: LaneId, message: Message) -> Result<MessageNonce>


// 메시지 증명 수신 (릴레이어 호출)
receive_messages_proof(
relayer_id: AccountId,
proof: MessagesProof,
messages_count: u32,
dispatch_weight: Weight
) -> Result<()>


// 전달 확인 증명 수신 (릴레이어 호출)
receive_messages_delivery_proof(
proof: MessagesDeliveryProof
) -> Result<()>
```

### 5.1.3 Parachains 팔렛 (modules/parachains/)

**목적**: 브릿지된 릴레이 체인의 하나 또는 여러 파라체인을 위한 라이트 클라이언트

**동작 원리**:

- 파라체인은 자체적으로 최종성을 달성할 수 없으므로 릴레이 체인의 최종성에 의존
- 릴레이 체인의 `paras` 팔렛에 있는 `Heads` 맵의 스토리지 증명을 검증
- GRANDPA 팔렛에서 가져온 릴레이 체인 헤더를 사용하여 증명을 검증

**핵심 기능**:

- 다중 파라체인 동시 추적 가능
- 파라체인별로 다른 프리미티브 사용 가능 (128비트, 32비트 블록 번호 등)
- 릴레이 체인 블록 번호를 사용한 파라체인 헤더 순서 관리

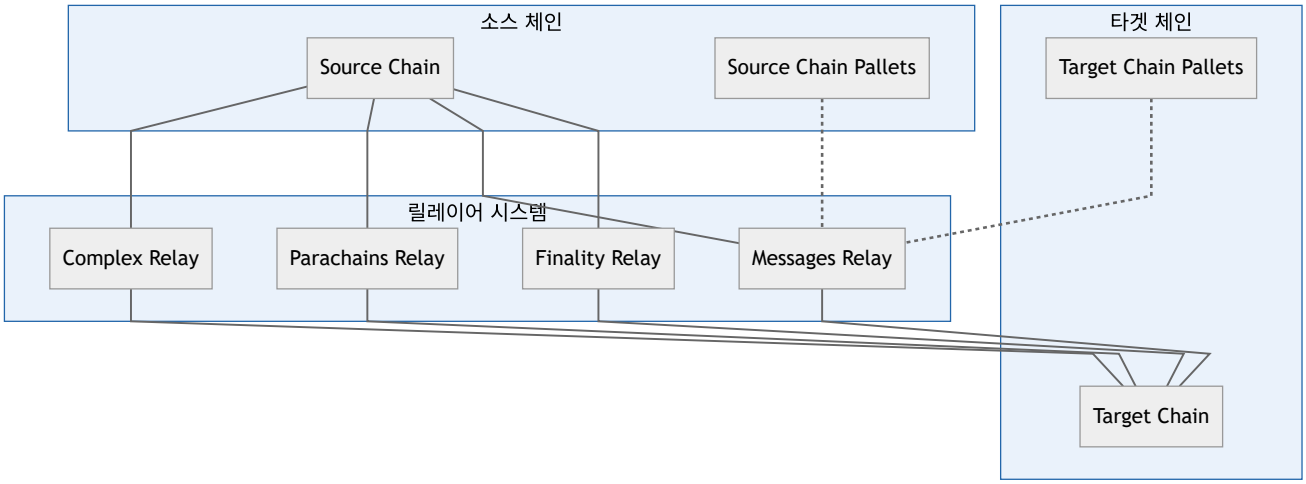
**주요 엔트리포인트**:

```
// 파라체인 헤드 제출
submit_parachain_heads(
proof: ParachainsProof,
parachains: Vec<(ParaId, ParachainHead)>,
relay_block: RelayBlockNumber
) -> Result<()>
```

# 릴레이어 시스템 구조 및 동작 원리

### 5.1.4 릴레이어 아키텍처 개요

릴레이어는 브릿지의 오프체인 컴포넌트로, 소스 체인과 타겟 체인에 모두 연결되어 상태를 동기화하는 역할을 합니다.




### 5.1.5 GRANDPA 최종성 릴레이어

**역할**: 소스 체인의 GRANDPA 정당화와 해당 헤더를 타겟 체인의 GRANDPA 라이트 클라이언트에 제출

**동작 과정**:

1. 소스 체인의 GRANDPA 정당화 스트림 구독
2. 새로운 정당화 감지 시 타겟 체인에 제출
3. 필수 헤더 검색 및 우선 제출
4. 검증자 세트 변경 추적

**특징**:

- 필수 헤더 제출은 수수료 면제
- 선택적 헤더 제출로 비용 최적화
- 자동 검증자 세트 업데이트

### 5.1.6 파라체인 최종성 릴레이어

**역할**: 소스 릴레이 체인의 파라체인 헤드를 타겟 체인에 동기화

**동작 과정**:

1. 소스 릴레이 체인의 `paras` 팔렛 `Heads` 맵 모니터링
2. 새로운 파라체인 헤드 감지 (블록 B에서)
3. 블록 B 또는 그 조상이 타겟 체인에 나타날 때까지 대기
4. 스토리지 증명 생성 및 타겟 체인에 제출

**의존성**:

- GRANDPA 최종성 릴레이어와 병렬 실행 필요
- 릴레이 체인 헤더의 최종성이 타겟 체인에 먼저 전달되어야 함

### 5.1.7 메시지 릴레이어

**구성**: 메시지 전달 릴레이어 + 전달 확인 릴레이어

**메시지 전달 릴레이어**:

1. 소스 체인의 아웃바운드 레인 엔드 모니터링
2. 새 메시지 큐잉 감지 (블록 B에서)
3. 블록 B 또는 그 후손이 타겟 체인에 나타날 때까지 대기
4. 메시지 스토리지 증명 생성 및 타겟 체인에 제출

**전달 확인 릴레이어**:

1. 타겟 체인의 인바운드 레인 엔드 모니터링
2. 새 메시지 전달 감지 (블록 B에서)
3. 블록 B 또는 그 후손이 소스 체인에 나타날 때까지 대기
4. 전달 확인 스토리지 증명 생성 및 소스 체인에 제출

### 5.1.8 복합 릴레이어 (Complex Relay)

**목적**: 비용 효율적인 온디맨드 릴레이 서비스 제공

**특징**:

- 메시지가 있을 때만 필요한 헤더를 릴레이
- 모든 필수 최종성 릴레이어를 온디맨드 모드로 실행
- 필수 GRANDPA 헤더는 여전히 자동 제출
- 메시지/확인이 필요할 때만 관련 블록 릴레이 요청

# 브릿지 보안 모델 및 신뢰 가정

### 5.1.9 보안 아키텍처

**라이트 클라이언트 기반 보안**:

- 전체 노드 실행 없이도 블록체인 상태 검증 가능
- GRANDPA 최종성 가젯의 암호학적 보안성 활용
- 2/3 + 1 검증자 합의 요구사항

**신뢰 가정**:

1. **소스 체인 최종성**: 소스 체인의 GRANDPA 최종성이 올바르게 작동
2. **검증자 정직성**: 소스 체인 검증자의 2/3 이상이 정직
3. **릴레이어 활성**: 최소 하나의 정직한 릴레이어가 활성 상태
4. **암호학적 가정**: 사용된 서명 스킴과 해시 함수의 안전성

### 5.1.10 공격 벡터 및 대응

**잠재적 공격**:

1. **Long Range Attack**: 오래된 검증자 세트를 사용한 공격
2. **Eclipse Attack**: 릴레이어의 네트워크 연결 차단

3. **Griefing Attack**: 잘못된 증명 제출로 인한 자원 낭비

4. **Censorship**: 특정 메시지나 헤더의 전달 차단

**보안 대응책**:

1. **체크포인트 시스템**: 정기적인 체크포인트로 롤백 제한

2. **다중 릴레이어**: 여러 독립적인 릴레이어 운영

3. **서명 확장**: 중복 헤더 제출 방지

4. **경제적 인센티브**: 정직한 행동에 대한 보상 시스템

### 5.1.11 거버넌스 및 업그레이드

**팔렛 소유자 시스템**:

- 모듈 레벨 sudo 계정 개념
- 런타임 업그레이드 없이 운영 중단/재개 가능
- 비상 상황 대응 메커니즘

**주요 거버넌스 기능**:

```
// 소유자 변경
set_owner(new_owner: AccountId)


// 운영 모드 설정 (정상/중단/제한)
set_operating_mode(mode: OperatingMode)


// 브릿지 초기화
initialize(init_data: InitializationData)
```

# 성능 최적화 및 확장성

## 5.1.12 트랜잭션 최적화

**서명 확장 (Signed Extensions)**:

- 중복 헤더 제출 자동 거부
- 트랜잭션 풀에서 무효 트랜잭션 제거
- 블록 공간 효율성 향상

**배치 처리**:

- 여러 메시지를 단일 트랜잭션으로 처리
- 파라체인 헤드 배치 제출
- 가스 비용 최적화

## 5.1.13 확장성 고려사항

**다중 레인 지원**:

- 애플리케이션별 독립적인 메시지 레인
- 병렬 메시지 처리 가능
- 레인별 독립적인 순서 보장

**모듈식 아키텍처**:

- 필요한 브릿지 컴포넌트만 선택적 배포
- 런타임별 맞춤형 구성 가능
- 새로운 브릿지 타입 쉽게 추가 가능

이 문서는 Polkadot SDK Bridges 시스템의 전체적인 아키텍처와 각 컴포넌트의 역할을 상세히 분석했습니다. 다음 문서에서는 Snowbridge의 구체적인 구현과 이더리움 브릿지의 특수한 요구사항들을 다룰 예정입니다.